

DAY 2

Module 5

Quality Gates Before Implementation

Catch problems while the artifacts are still cheap to change, not after code exists.

What You'll Learn

1

Generate and use a requirements-quality checklist

2

Run a cross-artifact consistency analysis before code is written

3

Budget token/cost usage and place security, compliance, and coverage checks as gates

Checklist: "Unit Tests for Requirements"

A checklist validates the spec itself (is it complete, clear, unambiguous, and consistent) before a single line of code exists.

Think of it as testing the requirement, not the implementation.

GATE TWO

Analyze: Cross-Artifact Consistency

Cross-checks spec, plan, and tasks against each other and against the constitution.

Catches drift, such as a task with no matching requirement or a plan choice that contradicts the spec, while it's still cheap to fix.

Token budget and compliance checks belong here too, not bolted on afterward.

Budgeting more tokens per feature up front is typically offset by far fewer rework cycles later.

Security, compliance, and code-coverage checks (e.g., SonarQube) are gates before implementation, not afterthoughts.

APPLYING THIS WITH

GitHub Spec Kit

Checklist and Analyze

`/speckit.checklist`

Generates a custom, reviewer-owned quality checklist for the feature. Run broadly or focused on one area.

`/speckit.analyze`

A read-only cross-artifact analysis across spec.md, plan.md, and tasks.md. Run before implementing, while artifacts are still cheap to adjust.

APPLYING THIS WITH

OpenSpec

A partial fit: some of this module's concept maps, some doesn't.

No Pre-Implementation Equivalent

No direct equivalent

OpenSpec has no checklist or analyze command that runs before code is written. Its closest command, `/opsx:verify`, exists, but it checks the implementation against the artifacts after `/opsx:apply`, not the artifacts against each other beforehand.

What to use instead

This is a genuinely different point in the loop, not a substitute for a pre-implementation gate. We'll come back to `/opsx:verify` in Module 6, where it belongs: reviewing what got built, not what's about to be built.

Module 5 at a Glance

	GitHub Spec Kit	OpenSpec
Pre-code quality gate	/speckit.checklist + /speckit.analyze	None (closest command runs after code)
Timing	Before implementation	/opsx:verify runs after /opsx:apply
Takeaway	Purpose-built pre-implementation gates	Gate the equivalent concern later, in Module 6

Key Takeaways

- Quality gates exist to catch problems while they're still cheap to fix.
- Spec Kit has purpose-built pre-implementation gates; OpenSpec's equivalent operates later in the loop.
- Token budget is a design decision, not just a cost line item.